Trackwise — How we ship

How a piece of work goes from an idea to production. It comes down to three habits:

1. **The product owner approves work before it starts.** Ideas are always welcome — but we build what's been approved, not just what we want to build.
2. `main` **only changes through a reviewed pull request.** No direct pushes, no self-merges.
3. **Nobody works alone.** Someone else understands every change before it lands — a feature is never seen for the first time at standup.

The branches

Two long-lived branches (never deleted); everything else is short-lived:

- `main` — always what's running in **production**. Only the release flow and hotfixes touch it.
- `staging` — the **test/staging environment**: a deployment running on its own Postgres database, where finished work is integrated and tested before it ships. Feature work merges here first.
- `feature/<name>`, `fix/<name>`, `chore/<name>` — short-lived branches off `staging`, merged back by pull request. A branch should land in days; too big means the task was too big — split it.
- `hotfix/<name>` — short-lived, branched off `main`, for emergencies only (see below).

Any further long-lived branches follow the same pattern: one deployment, one Postgres database, one purpose.

The path of a piece of work

```
task (approved) ⟶ feature branch ⟶ PR to staging ⟶ QA tests on staging
        ⟶ release PR (staging → main) ⟶ merge ⟶ production
```

1. It starts as a task in Trackwise

We track our own work in the app — it's our tracker.

- Every feature or fix starts as a task in Trackwise saying **what** and **why**.

- The product owner approves it through the app's sign-off flow before work begins: an unapproved task sits as a proposal until they sign it. Have an idea? Add the task and let the owner decide — if it's a yes, it's a small wait.
- A task is one piece of work — if it needs more than one PR, split it into separate tasks.
- When you start work, open the task and use **Update progress** to set its status to in progress — that's the status the team sees. If you're not on the task yet, only a leader or the task's creator can add you — just ask one of them.

2. Branch off `staging` and build

- `git checkout -b feature/<name>` from `staging`.
- Commit messages say what and why (`Let stakeholders complete goals`, not `update`). The squash-merge title becomes the changelog, so keep it meaningful.
- Before asking for review: `npm run typecheck`, `npm run lint`, `npm run test:unit`, `npm run build`, and the walkthrough for any flow you touched.
- Changed a convention or decision? Update `AGENTS.md` or the `docs/PLAN.md` log in the same PR.
- **Post progress updates on the task as you work** (Update progress — status plus a note). That's where the team looks to see where it stopped and what's next; blocked means blocked with a reason, not silent.

3. Pull request into `staging`

In short:

- Link the Trackwise task it delivers.
- Ask one teammate for review, someone who didn't work on it. Rotate reviewers so everyone stays familiar with the codebase.
- Write a description that teaches: what changed, why, what to look at.
- Merge when checks are green and the review is approved — **squash-merge** here keeps `staging` history readable. The author merges.
- Merging to `staging` deploys the staging environment automatically. If the PR changed `db/schema.ts`, run `npm run db:push` against the staging Postgres database first (below).

4. Test on staging (QA)

Merging to `staging` deploys the staging environment — a full copy of the app running on its own database, so testing never touches production data. This stage is testing, not a look:

- **QA exercises the feature on staging the way a user would** — the real flow, not just the happy path. Bugs found here go back through the normal path: a new branch and PR into

`staging`, never a quick tweak on the branch itself.

- The **author** confirms their change behaves on staging; the **product owner** signs off on user-facing work there. This is where a feature is first seen working — not at standup.
- Every PR still gets its own preview deployment for early looks before merge, but staging is where testing and sign-off happen.

5. Ship it: the release PR

- When work on staging is verified and ready, open a pull request **`staging`** → **`main`** — the release PR. It lists everything going out, the owner approves it, and the release owner merges it.
- Use a **merge commit, not squash**, for `staging` → `main`: squashing rewrites the commits and the two branches drift, so every later release fights phantom conflicts.
- If the release includes schema changes, run `npm run db:push` against the production Postgres database immediately before merging (below). Merge, watch production, mark the tasks done.

Hotfixes: when production is broken

For a severe bug that's broken the live app:

- Branch off `main` as `hotfix/<name>` — `main` is always what's running in production, so you're already fixing the exact broken version. No waiting for the next release.
- Fix, run the checks, open the PR marked as a hotfix. It's still a PR — but expedited: one teammate reviews, the owner is notified on the task, merge as soon as it's green.
- After it lands, **merge `main` back into `staging`** so the fix exists on both long-lived branches and the next release doesn't reintroduce the bug.
- Verify the fix on production right away and note what happened on the task.
- A hotfix only patches the emergency — any cleanup afterwards goes through the normal flow.

Environments

Environment	What it is	Database
Local	`npm run dev` on your machine	your own Postgres database — a Neon branch off staging
Preview	a live deployment built automatically for each pull request	shares the staging Postgres database

Environment	What it is	Database
Staging (test)	the deployment of `staging` — QA tests and the owner signs off here before release	the staging Postgres database
Production	the deployment of `main` — the real site	the production Postgres database

Every environment is a full deployment of the app pointed at its own Postgres database — nothing on staging or a preview ever reads or writes production data.

The app is hosted on Vercel: each long-lived branch deploys to its environment on merge, and each pull request gets a preview URL automatically. GitHub Actions runs the checks (typecheck, lint, unit tests, build) on every PR.

Email is sent by a small mail service the app calls over HTTPS (`MAIL_SERVICE_URL` / `MAIL_SERVICE_TOKEN`); it isn't part of these deployments.

Changing the database

The schema lives in `db/schema.ts` and is applied with `npm run db:push`. Each environment has its own Postgres database, so a change travels the same path as the code:

- **Local:** everyone develops against their own database — a Neon branch off the staging database. Breaking and experimental schema changes live there and never reach a shared database.
- **Shared databases (staging and production):** `db:push` is run **manually, immediately before the merge** — deliberate and auditable, not automatic. On staging the merger runs it; on production the release owner runs it before the release PR merges, noted on the PR. If a PR's preview needs the new schema to render at all, push earlier.
- **Additive only** on shared databases: new tables, columns and indexes, nothing removed, renamed or retyped. A breaking change is a rare, planned event (backup → migrate → push), agreed with the team first.
- **Merge latest `staging` before pushing to a shared database.** `db:push` diffs your `schema.ts` against the live database — if a teammate pushed first, it will offer to *drop* their work. If it proposes deleting anything you didn't write, abort and merge.
- If a build fails, fix the code — never reshape the database to make it pass.

Setup (one-time, repo admin)

Settings → Branches → protect **`main`** and **`staging`**:

- Require a pull request (1 approving review, stale reviews dismissed)
- Require the `Trackwise quality checks` status check to pass
- Require conversations resolved and branches up to date before merging
- Apply the rules to everyone, admins included

On the Trackwise side, the owner's Proposals page is the queue of work waiting for a decision — that's where "what are we building, and who asked for it" stays visible to the whole team.

Keeping everyone in the loop

- Reviewers rotate — everyone reviews everyone's code within a few weeks.
- The first time anyone sees a feature working is its deployment on staging (or its PR preview), not a surprise demo.
- Ask on the task or the PR rather than guessing — the discussion stays with the work.